BTS CIEL – Cybersécurité, Informatique et réseaux, Électronique
Spécialité Informatique et Réseaux (IR)

DOSSIER TECHNIQUE DE PROJET

Partie Individuelle – Revue Intermédiaire

Projet	Accès Campus
Rôle	Étudiant 2 – PSW Interface Client (Frontend)
Candidat	Mathis Blanchard
Établissement	Lycée Saint Aubin La Salle
Date de revue	Mars 2026

Sommaire

1. SITUATION DANS LE PROJET **2**
1.1 – Synoptique de la réalisation 2
1.2 – Description de la partie personnelle 3
2. RÉALISATION DES TÂCHES ET CHOIX TECHNIQUES **5**
2.1 – Choix technologiques 5
2.2 – Conception détaillée 6
3. BILAN DE LA RÉALISATION PERSONNELLE **12**

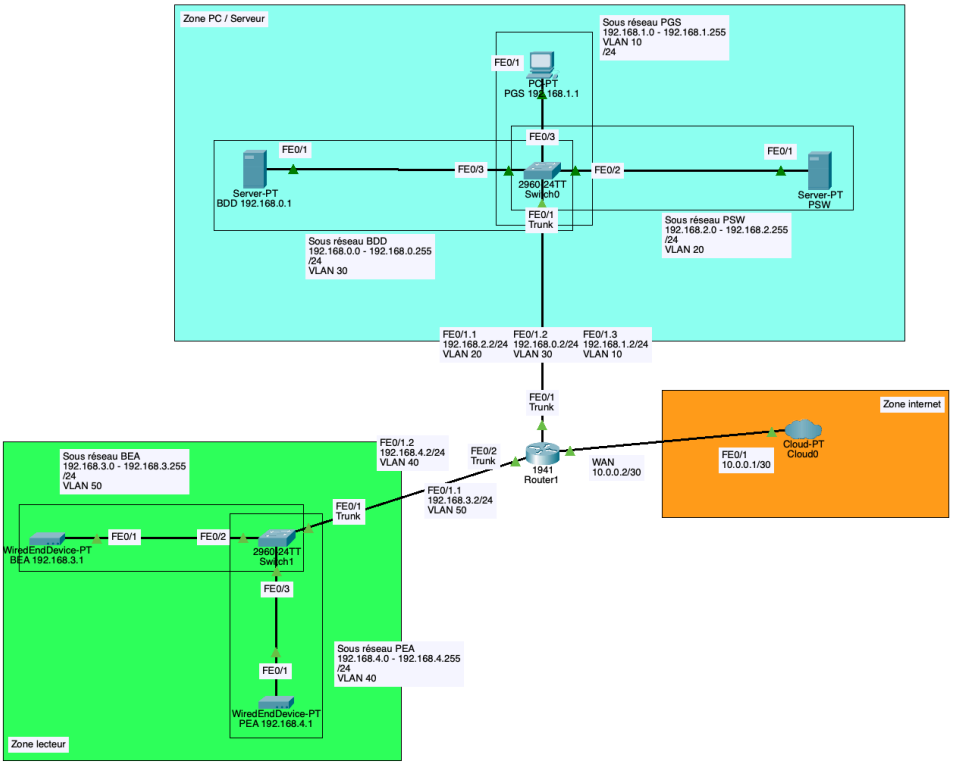
1. SITUATION DANS LE PROJET

1.1 – Synoptique de la réalisation

Le projet « Accès Campus » vise à moderniser le contrôle d'accès et la gestion des salles du lycée Saint Aubin La Salle. Il repose sur un écosystème connecté composé de quatre nœuds matériels et logiciels distincts, qui communiquent entre eux au sein d'une infrastructure réseau segmentée par VLANs :

- **PEA – Poignée Électronique Autonome :** dispositif installé sur chaque porte de salle, intégrant un lecteur de badge RFID pour contrôler physiquement l'accès.
- **BAE – Borne d'Appel Étudiant :** borne tactile déployée dans les classes. Elle permet aux élèves de badger, de consulter leur emploi du temps sans passer par un poste fixe.
- **PGS – Poste de Gestion des Salles :** poste dédié au personnel administratif pour gérer les droits d'accès, les plannings et les utilisateurs du système.
- **PSW – Poste Serveur Web :** cœur du système. Il héberge l'Interface Client Web (Frontend) dont je suis responsable. Il expose un « Client Léger » accessible depuis n'importe quel navigateur web, aussi bien sur les postes du campus que sur smartphones.

Sur le plan réseau, le serveur PSW est matérialisé par « Serveur-PT PSW » situé dans la zone PC/Serveur. Il est rattaché au Sous-réseau PSW, dont la plage d'adressage est 192.168.2.0/24 et qui correspond au VLAN 20. Ce VLAN est isolé du réseau pédagogique (VLAN utilisateurs) par un routeur jouant le rôle de pare-feu filtrant les flux HTTP via des règles ACL. Seul le port 80 est autorisé en entrée sur ce VLAN, ce qui réduit la surface d'attaque au strict minimum.



1.2 – Description de la partie personnelle

Dans ce projet, j'assure le rôle d'Étudiant 2. Ma mission couvre exclusivement le développement du Frontend de l'application web hébergée sur le PSW. Conformément à la contrainte d'exigence Id 8 du cahier des charges, cette interface doit être accessible depuis un navigateur standard, que ce soit sur poste fixe ou sur smartphone, sans aucune installation côté client (principe du « Client Léger »).

Les quatre fonctionnalités que je dois implémenter sont les suivantes :

- Visualiser les disponibilités des salles en temps réel.
- Visualiser la présence des étudiants dans une salle donnée.
- Rechercher une salle libre pour une période donnée.
- Afficher les informations de présence détaillées pour une salle.

La chronologie de mon travail jusqu'à cette revue intermédiaire est la suivante :

1. Analyse du cahier des charges et lecture des diagrammes d'exigences (SysML). J'ai identifié les fonctions attendues du Frontend et traduit les exigences métier en spécifications techniques exploitables.
2. Prototypage des interfaces utilisateurs (Mockups) sur le logiciel Figma. Cette phase m'a permis de définir visuellement l'ensemble des écrans (page d'accueil, planning des salles, disponibilité, annuaire) avant de coder quoi que ce soit.
3. Rédaction des scénarios d'utilisation et modélisation des diagrammes de séquence UML, diagramme d’exigence. Par exemple, le scénario « Réservation de salles V1.1 » (RS1) décrit pas à pas l'enchaînement des échanges entre l'interface, le serveur et la base de données.
4. Étude et validation des choix technologiques. J'ai rédigé un dossier de choix techniques documentant les arguments retenus pour chaque brique du Frontend (voir section 2.1).
5. Début du développement web et de la configuration du serveur local. La structure HTML/CSS est en cours d'intégration et le serveur Nginx est configuré sur la machine Debian de test.

2. RÉALISATION DES TÂCHES ET CHOIX TECHNIQUES

2.1 – Choix technologiques

L'ensemble des choix techniques a été consigné dans mon dossier de choix techniques et validé avec l'équipe projet. Ils s'articulent autour des critères suivants : performance, coût nul (Open Source), maintenabilité et adéquation avec les contraintes du cahier des charges et que j'ai donc réduit pour que ça donne un tableau plus visible et compréhensible.

Couche	Technologie retenue	Justification principale
Serveur & OS	Debian GNU/Linux + Nginx	Faible empreinte mémoire, point d'entrée HTTP unique et performant
Architecture	Séparation Frontend / Backend	Le Client Web (Frontend) est séparé de l'API C++ (Backend)
Communication	API Fetch JS + HTTP/REST + JSON	Requêtes asynchrones (AJAX) sans rechargement de page ; JSON léger et universel
Langages Frontend	HTML, CSS, Vanilla JS + Vue.js	Responsive Design multi-support ; Vue.js pour la réactivité de l'interface
Outils	Figma, Visual Studio Code	Prototypage rapide (Figma) ; IDE léger et extensible (VSCode)
Coût	100 % Open Source	Respect de la contrainte d'exigence Id 8.1 (coût = 0 €)

Je détaille ci-dessous les justifications :

- **Serveur & Hébergement (Debian + Nginx) :** La machine cible est dimensionnée au juste nécessaire (2 cœurs, 4 Go de RAM, 32 Go de stockage), car la charge applicative attendue reste modérée (consultation de disponibilités de salles). Nginx a été retenu face à Apache car il consomme significativement moins de mémoire en mode statique et agit comme un point d'entrée HTTP unique, faisant office de reverse proxy vers l'API C++. Ce choix simplifie également la configuration du pare-feu.

- **Séparation Frontend / Backend :** Mon binôme développe l'API en C++ (framework Drogon). Cette architecture séparée impose que nos deux composants communiquent via un contrat d'interface formalisé, ce qui présente plusieurs avantages : chacun peut développer et tester sa partie indépendamment, et il est possible à terme de remplacer l'un des composants sans impacter l'autre.

- **API Fetch JS + JSON (communication asynchrone) :** J'utilise l'API Fetch native de JavaScript pour effectuer des requêtes HTTP asynchrones vers l'API Backend. Cela signifie que l'interface peut interroger le serveur pour obtenir la liste des salles disponibles sans recharger entièrement la page, offrant une expérience fluide à l'utilisateur. Le format JSON a été retenu pour son universalité, sa légèreté et sa facilité de parsing côté JavaScript. Exemple de réponse attendue : { "salle": "B202", "occupé": true, "etudiants": 24 }.

- **Vue.js (framework JavaScript) :** Au-dessus de JavaScript natif (Vanilla JS) pour la logique de base, j'utilise Vue.js pour rendre l'interface réactive et dynamique. Vue.js permet de lier directement les données JSON reçues du serveur aux éléments HTML affichés, sans écrire de manipulations DOM manuelles fastidieuses. C'est un framework léger qui n'alourdit pas les temps de chargement.

- **Coût total (contrainte exigence Id 8.1) :** L'intégralité de la stack (Debian, Nginx, HTML/CSS/JS, Vue.js, VS Code, Figma en version gratuite) est Open Source ou gratuite. Le coût de licence est de 0 €, ce qui respecte strictement la contrainte budgétaire du cahier des charges.

2.2 – Conception détaillée

La conception de mon Frontend a suivi une démarche structurée en deux temps : d'abord la définition visuelle via les maquettes Figma, puis la modélisation comportementale via les diagrammes de séquence UML.

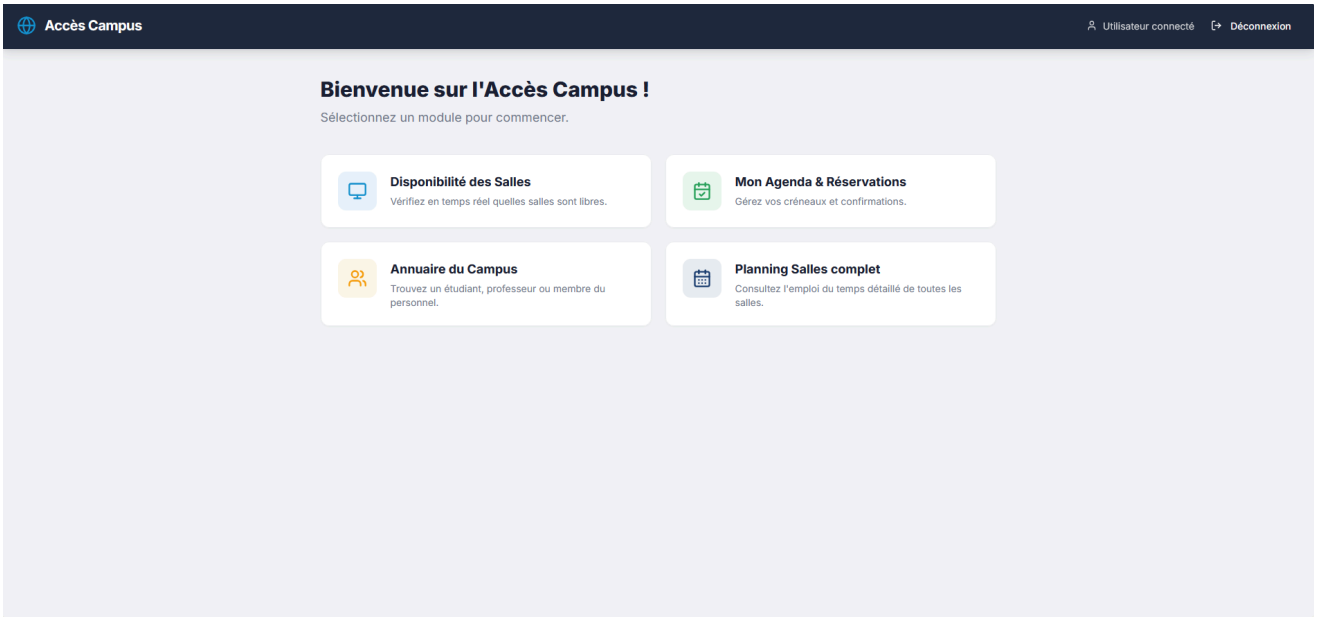
2.2.1 – Interfaces utilisateurs (Maquettes Figma)

J'ai réalisé l'ensemble des maquettes sur Figma avant de commencer le développement. Cette démarche m'a permis d'une part de valider la logique de navigation, et d'autre part de définir avec mon binôme : quelles données l'API doit renvoyer, dans quel format JSON, pour que chaque écran puisse s'afficher correctement.

L'application web est structurée autour des écrans suivants :

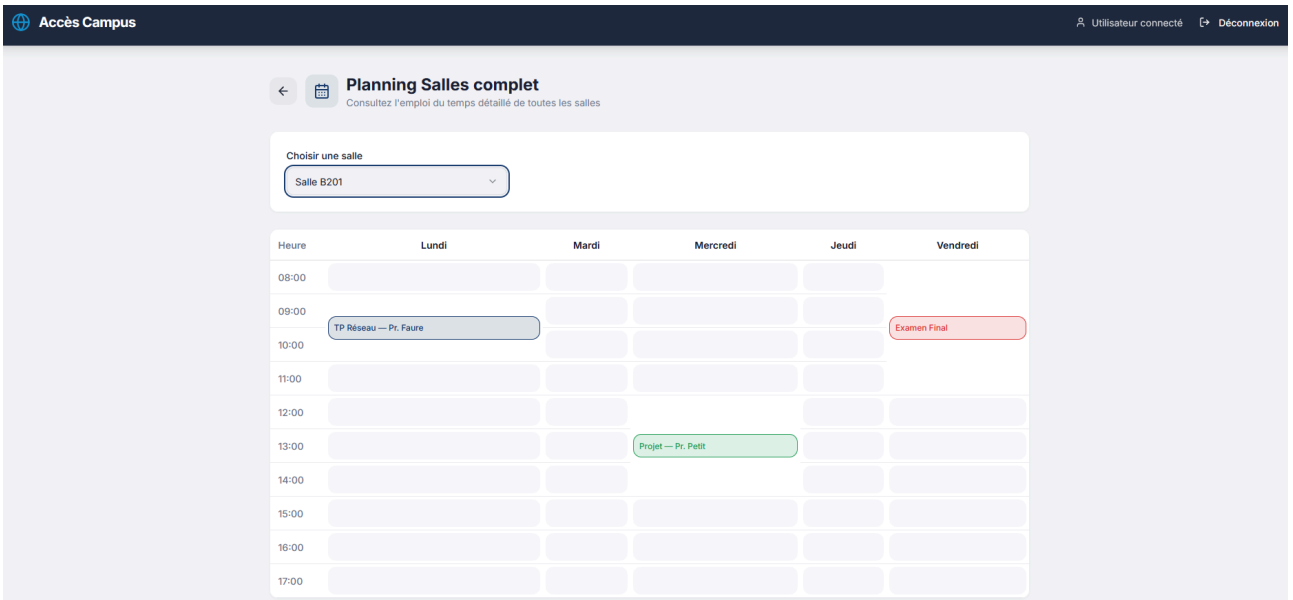
Page d'accueil

Point d'entrée de l'application. Elle présente les fonctionnalités disponibles sous forme de tuiles cliquables et affiche un récapitulatif de l'état global des salles (nombre de salles libres/occupées en ce moment).



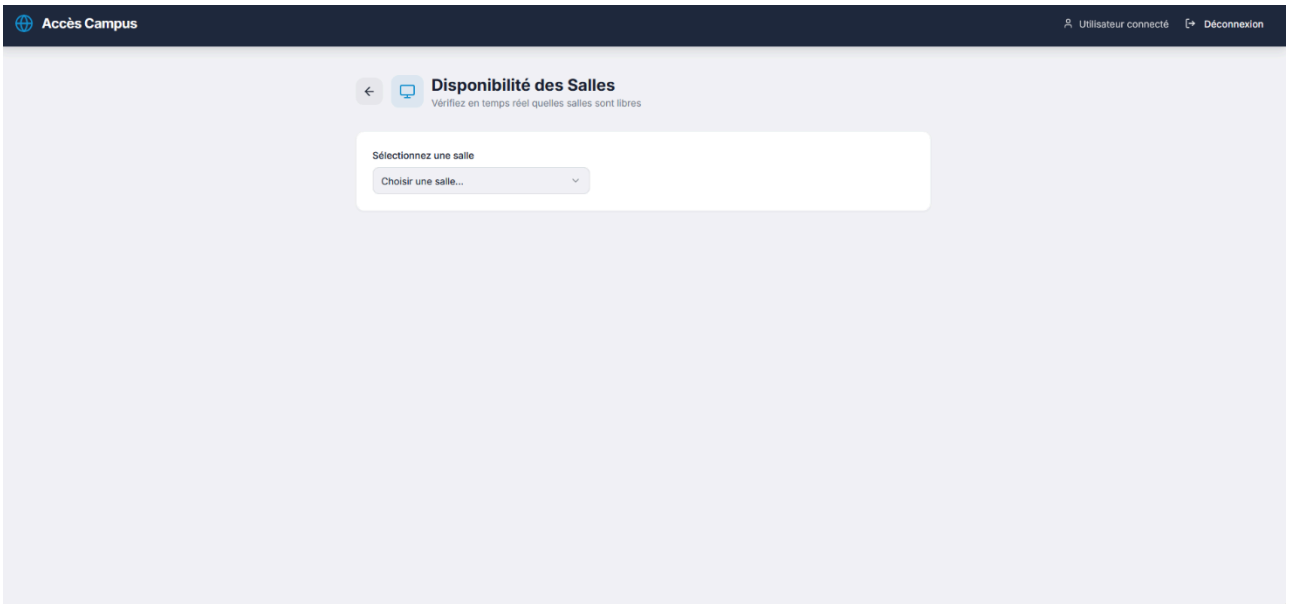
Planning des salles

L'utilisateur sélectionne une salle dans une liste déroulante. L'interface affiche immédiatement sa grille horaire (créneaux libres en vert, occupés en rouge) sans rechargement de page grâce à Vue.js.



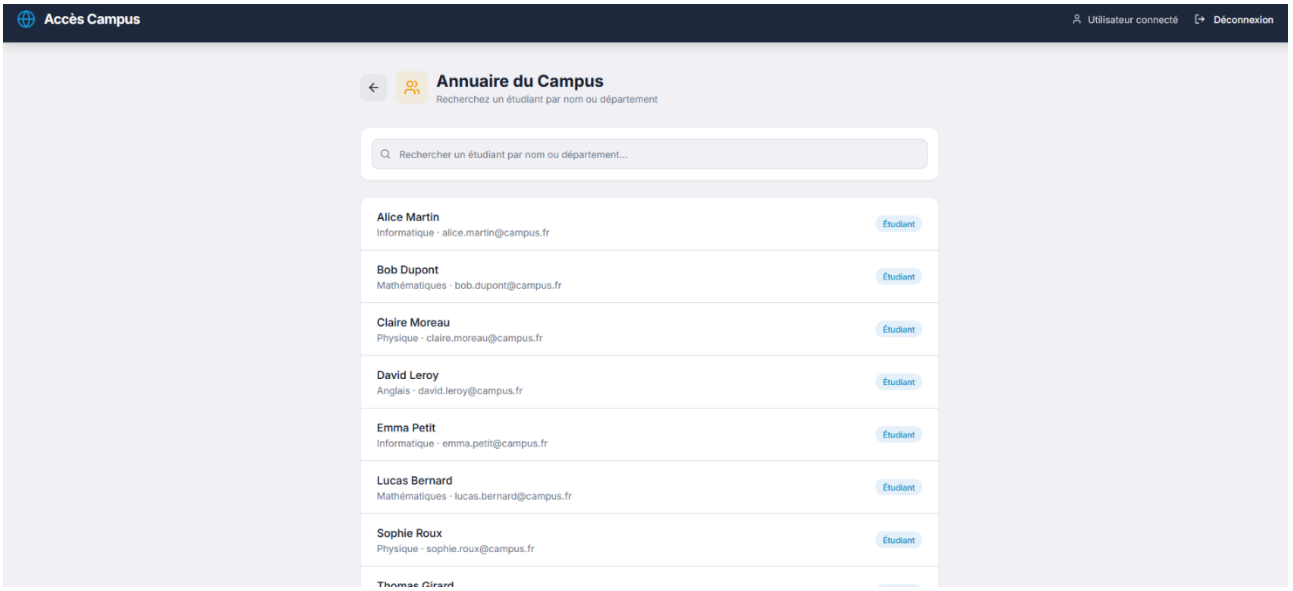
Disponibilité des salles

L'utilisateur saisit une date, une heure de début et une durée. Après vérification de la saisie côté client, l'interface interroge l'API et affiche la liste des salles disponibles pour ce créneau. Un message explicite s'affiche si aucune salle n'est libre.



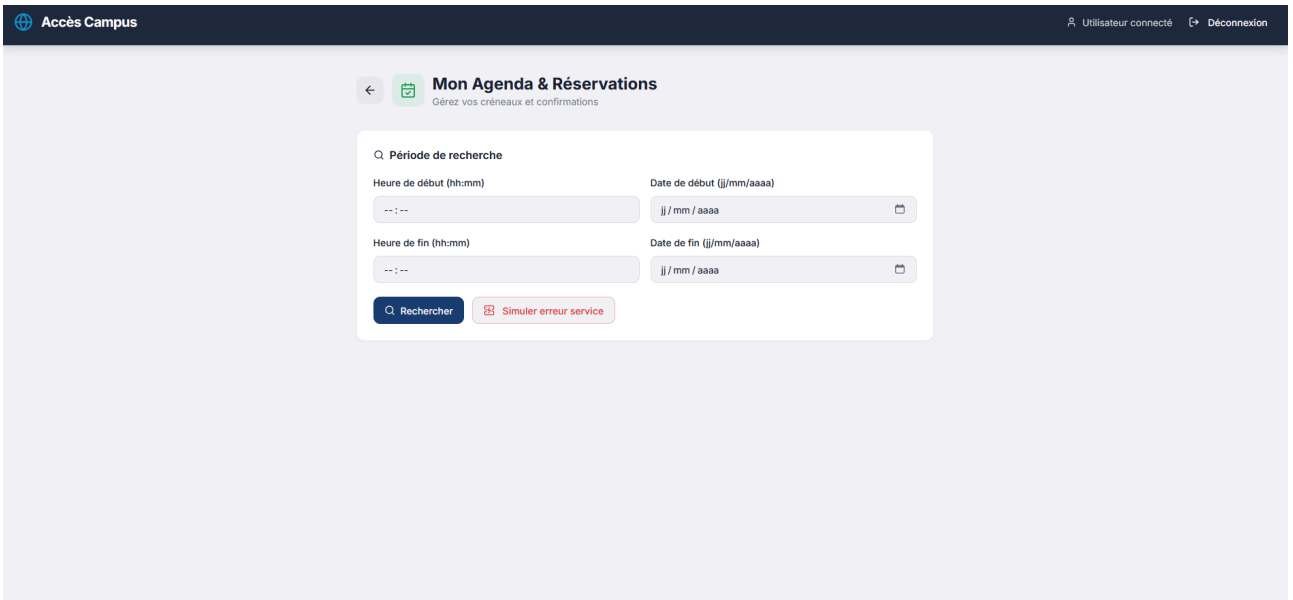
Annuaire / Présence en salle

Deux modes d'utilisation : consulter la liste des étudiants présents dans une salle donnée (nom, prénom, heure d'entrée), ou rechercher un étudiant pour accéder à son historique de présences.



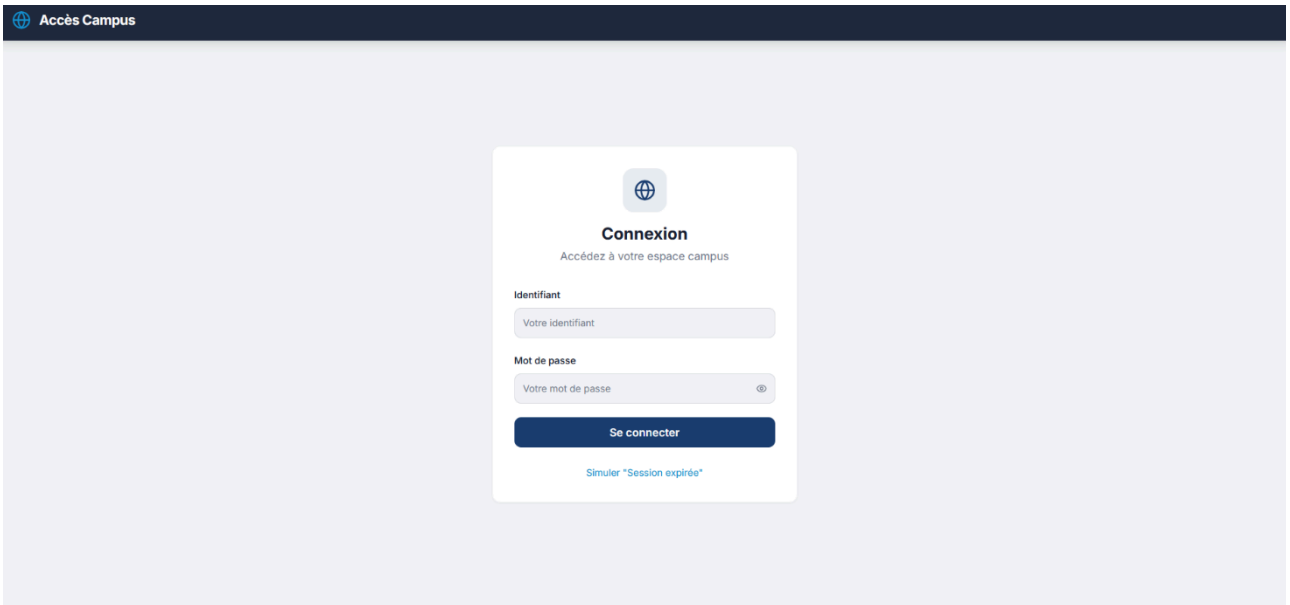
Agenda et Réservation

Permet de réserver une salle en indiquant l'heure de début, date de début puis heure de fin et date de fin avec comme format (hh :mm), (jj/mm/aaaa).



Page de connexion

Cette page permet tout simplement de saisir ses identifiants et de s'identifier.



Toutes ces interfaces respectent les principes du Responsive Design ce qui garantit un rendu adapté aussi bien sur grand écran que sur smartphone ou tablette.

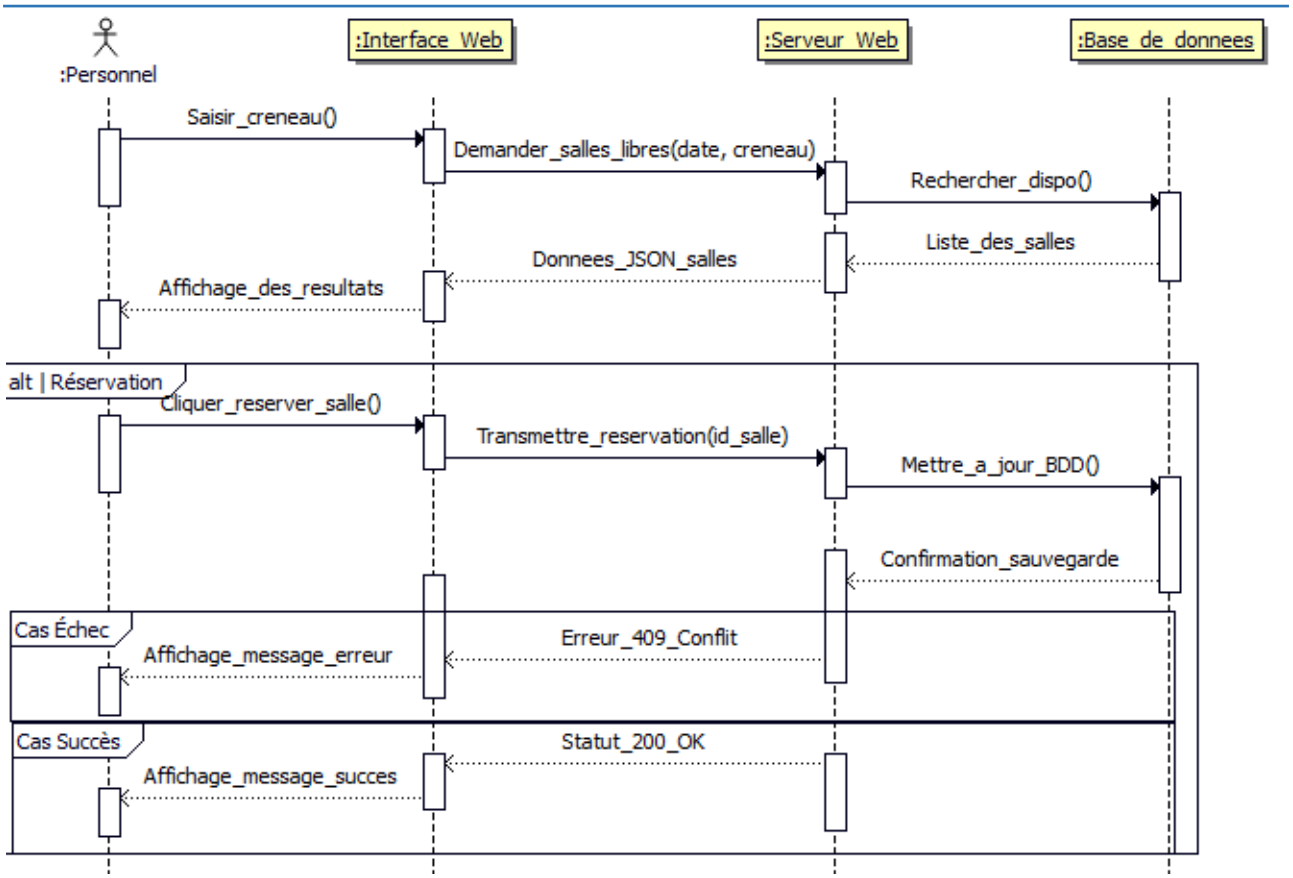
2.2.2 – Cas d'utilisation : Recherche de salle disponible (Scénario RS1)

Je vais détailler ici le scénario d'utilisation le plus représentatif de mon périmètre : la recherche et la visualisation des disponibilités de salles (Scénario RS1, version 1.1). Ce scénario illustre la cinématique complète d'une interaction, du clic de l'utilisateur jusqu'à l'affichage du résultat.

Acteurs impliqués : (enseignant ou administratif), Interface Client Web (Frontend – ma partie), Serveur Web / API C++ (Backend – partie binôme), Base de données PostgreSQL (VLAN 30).

Cinématique du diagramme de séquence :

6. L'utilisateur (Personnel) sélectionne une date, une heure de début et une durée souhaitée dans le formulaire de recherche de l'interface.
7. Avant d'envoyer la requête, l'interface effectue une vérification de cohérence côté client (ex. : la date de fin ne peut pas être antérieure à la date de début, les champs ne peuvent pas être vides). Si la saisie est invalide, un message d'erreur est affiché immédiatement sans solliciter le serveur.
8. Si la saisie est valide, l'interface construit une requête HTTP GET vers l'API.
9. Le serveur de mon binôme reçoit la requête, son API C++ (Drogon) interroge la base de données PostgreSQL via une requête SQL filtrée, et retourne la liste des salles correspondantes au format JSON.
10. L'interface JavaScript/Vue.js reçoit la réponse JSON et met à jour dynamiquement la liste des salles affichées à l'écran, sans rechargement de la page.
11. En cas d'erreur serveur (HTTP 404 – aucune salle trouvée, ou HTTP 409 – conflit de réservation simultanée), l'interface affiche un message explicite adapté au code d'erreur reçu.


:Personnel
:Interface_Web
:Serveur_Web
:Base_de_donnees
Saisir_creneau()
Demander_salles_libres(date, creneau)
Rechercher_dispo()
Liste_des_salles
Donnees_JSON_salles
Affichage_des_resultats
alt | Réservation
Cliquer_reserver_salle()
Transmettre_reservation(id_salle)
Mettre_a_jour_BDD()
Confirmation_sauvegarde
Cas Échec
Erreur_409_Conflit
Affichage_message_erreur
Cas Succès
Statut_200_OK
Affichage_message_succes

3. BILAN DE LA RÉALISATION PERSONNELLE

Cette revue se tenant à une étape intermédiaire du projet, je dresse ici un bilan honnête de l'avancement de mes travaux, en distinguant ce qui est validé de ce qui reste à produire.

Tâche / Livrable	État	Remarque
Analyse des exigences (SysML)	Terminée	Diagrammes d'exigences lus et annotés
Prototypage Figma (Mockups)	Terminée	Maquettes validées
Scénarios d'utilisation & diagrammes de séquence	Terminés	Scénario RS1 modélisé en UML
Choix technologiques (Vue.js, Nginx, JSON)	Terminé	Dossier de choix techniques finalisé
Structure HTML/CSS	En cours	Intégration des maquettes en code
Développement JS/Vue.js (logique métier)	À faire	Liaison interface ↔ données réelles
Tests unitaires (Fetch, HTTP 404/409/200)	À faire	Validation des cas d'erreur
Tests d'intégration avec l'API C++	À faire	Validation du contrat JSON avec le binôme

Points validés

- **Analyse UML/SysML terminée :** L'étude du cahier des charges et la lecture des diagrammes d'exigences m'ont permis de bien cerner le périmètre fonctionnel de ma partie. Cette phase est close.
- **Maquettes Figma validées :** L'ensemble des écrans a été prototypé et validé.
- **Choix technologiques actés :** Vue.js, Nginx, JS, CSS, Html, JSON, Debian, Figma, VS Code. Ces choix sont documentés et justifiés dans le dossier de choix techniques.
- **Architecture réseau définie :** La topologie réseau (VLANs, adressage IP, règles ACL) est entérinée en accord avec le reste de l'équipe.

Parties restant à développer

- **Finalisation du code JavaScript/Vue.js :** La structure HTML/CSS est en cours d'intégration, mais la logique métier (appels API Fetch, binding des données Vue.js, gestion des états) reste à implémenter pour lier l'interface aux données réelles servies par l'API C++.
- **Tests unitaires :** Je devrai écrire des tests pour valider le comportement de mes requêtes Fetch dans tous les cas : succès (HTTP 200), ressource non trouvée (HTTP 404) et conflit de réservation simultanée (HTTP 409). Ces tests garantiront la robustesse de l'interface face aux aléas réseau et serveur.
- **Tests d'intégration avec l'API C++ :** Une fois les deux composants suffisamment avancés, mon binôme et moi devrons procéder à des tests d'intégration conjoints pour valider l'ensemble de la chaîne : Frontend → Nginx → API C++ → PostgreSQL → réponse JSON → affichage.

Conclusion personnelle

Cette revue intermédiaire me confirme que la phase de conception et d'analyse, bien que souvent perçue comme abstraite, est en réalité le fondement de tout le reste. Les heures investies dans les maquettes Figma et les diagrammes de séquence UML n'ont pas été du temps perdu : elles m'ont permis de commencer le développement avec une vision claire de ce que je dois produire et de bien comprendre le projet.

Cela m'a également appris l'importance critique du travail d'équipe dans un projet système. La définition rigoureuse du contrat d'interface JSON avec mon binôme Backend est l'exemple le plus concret de cette interdépendance.

Pour la suite, je me concentrerai en priorité sur l'implémentation de la logique JavaScript/Vue.js, puis sur les tests unitaires, de façon à livrer un Frontend fonctionnel et testé en vue de la prochaine revue.

Mathis Blanchard – Étudiant 2, BTS CIEL IR

Projet Accès Campus – Lycée Saint Aubin La Salle – Mars 2026